

PRÀCTICA 2: PROCESSOS SÍNCRONS

Jordi Garcia
Mateu Magem

1. INTRODUCCIÓ

L'objectiu de la pràctica consisteix en incloure el temps en el nostre procés a partir de la manipulació d'un procés sincron. Per això, utilitzarem un senyal de clock per realitzar una transferència de dades d'un registre a un altre en un instant determinat.

La primera part de la pràctica consisteix en mostrar un valor diferent en cada un dels 4 displays de la placa a partir dels 16 interruptors d'aquesta (4 interruptors per cada display). El problema d'això és que només disposem de 8 càtodes pels 4 displays, per tant, haurem de llegir els interruptors (activant l'ànode corresponent) amb una freqüència que l'ull humà no sigui capaç de veure.

2. PROJECTE I DESCRIPCIÓ DE L'ARQUITECTURA

Primer de tot hem de crear el projecte, seguirem els mateixos passos que vam seguir en la pràctic anterior. Aquí la diferència estarà quan ens demani que definim l'entitat amb els ports d'entrada i sortida ja que en aquest cas, disposarem del senyal de rellotge.

Així doncs, l'entitat ens quedarà d'aquesta manera:

```
entity p2_sincron is
    Port ( sw : in STD_LOGIC_VECTOR (15 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
end p2_sincron;
```

Un cop amb l'entitat definida toca crear el cos del nostre procés, l'arquitectura.

Seguidament descriurem els passos que hem seguit per dur-la a terme.

Primer de tot programarem un divisor de clock. Això ho farem perquè amb una freqüència de 1kHz ens és suficient pel nostre procés.

```
-- Divisor del clk (de 100Mhz a 1kHz)
divisor_clk: process (clk, reset) begin
    if clk'event and clk = '1' then
        div_clk <= '0';
        if reset = '1' then
            div_cnt <= 0;
        elsif div_cnt = 100000 then
            div_cnt <= 0;
            div_clk <= '1';
        else div_cnt <= div_cnt + 1;
        end if;
    end if;
end process;
```

També crearem un comptador que compti de 0 a 3 a la freqüència del divisor de clock. Els valors d'aquest comptador ens serviran per relacionar els grups de 4 interruptors amb els ànodes que volem activar.

```
-- Comptador que assigna la entrada que toca
digit_actual: process(clk, reset) begin
if clk'event and clk = '1' then
    if reset = '1' then
        digit_act <= 0;
    elsif div_clk = '1' then
        if digit_act = 3 then
            digit_act <= 0;
        else digit_act <= digit_act + 1;
        end if;
    end if;
end if;
end process;
```

Un cop hem creat la eina que relaciona els ports d'entrada amb els de sortida toca programar aquestes relacions.

Primer de tot, crearem una nova variable anomenada sg que guardarà el grup de 4 interruptors que hem de llegir en cada cas.

```
-- Valors dels catodes
selector: process(digit_act, sg, sw) begin
-- Assignar conjunt de switches a sg
case digit_act is
    when 0 =>
        sg <= sw(3 downto 0);
    when 1 =>
        sg <= sw(7 downto 4);
    when 2 =>
        sg <= sw(11 downto 8);
    when 3 =>
        sg <= sw(15 downto 12);
end case;
end process;
```

Farem el mateix pels ànodes.

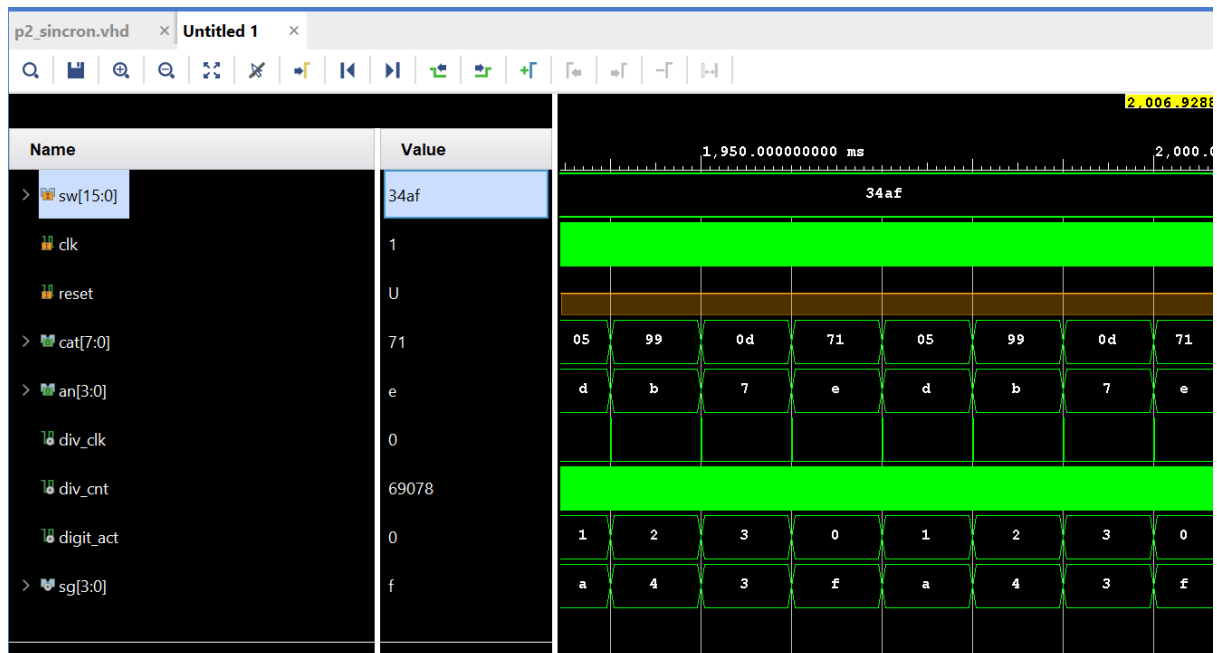
```
-- Assignar ànode
with digit_act select an <=
    "1110" when 0,
    "1101" when 1,
    "1011" when 2,
    "0111" when 3;
```

D'aquesta manera, activarem el display 0 quan llegim el primer grup de 4 interruptors i així successivament amb la resta d'interruptors i displays.

Per últim, només ens quedarà la funció que en funció de sg (grup de 4 interruptors) dona el valor corresponent als càtodes, en aquest cas, per mostrar el número en hexadecimal

```
-- Assignar grup de switches al càtode
with sg select cat <=
    "00000011" when "0000", --0
    "10011111" when "0001", --1
    "00100101" when "0010", --2
    "00001101" when "0011", --3
    "10011001" when "0100", --4
    "01001001" when "0101", --5
    "01000001" when "0110", --6
    "00011111" when "0111", --7
    "00000001" when "1000", --8
    "00011001" when "1001", --9
    "00000101" when "1010", --a
    "11000001" when "1011", --b
    "11100101" when "1100", --c
    "10000101" when "1101", --d
    "01100001" when "1110", --E
    "01110001" when "1111", --F
    "11111111" when others;
```

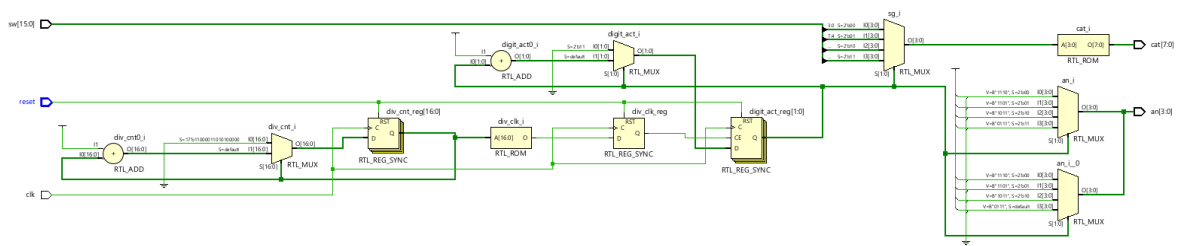
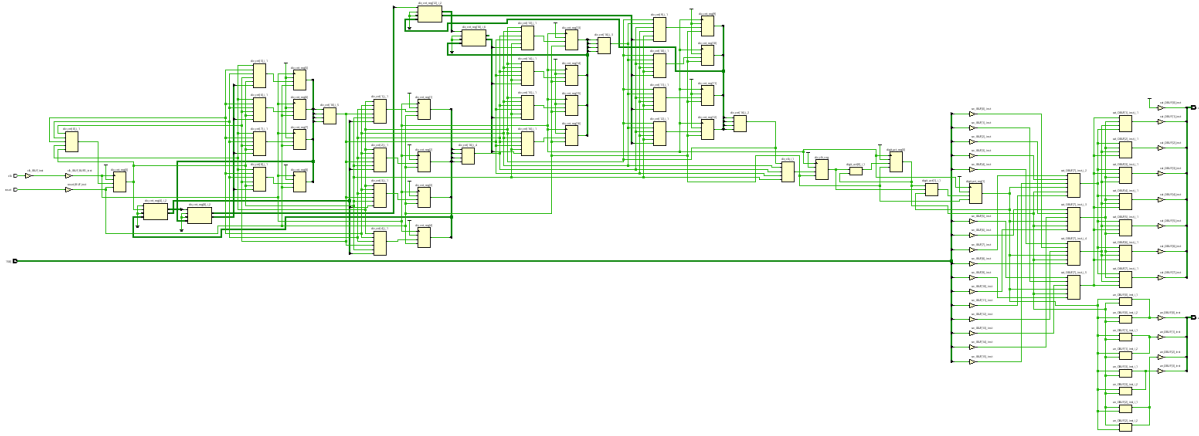
3. SIMULACIÓ



Aquí tenim l'exemple d'un tros de simulació forçant un valor als interruptors i un clock. Veiem com el digit_act va contant i mostrant els diferents valors de sg per cada valor d'aquest. Fa el mateix amb els ànodes.

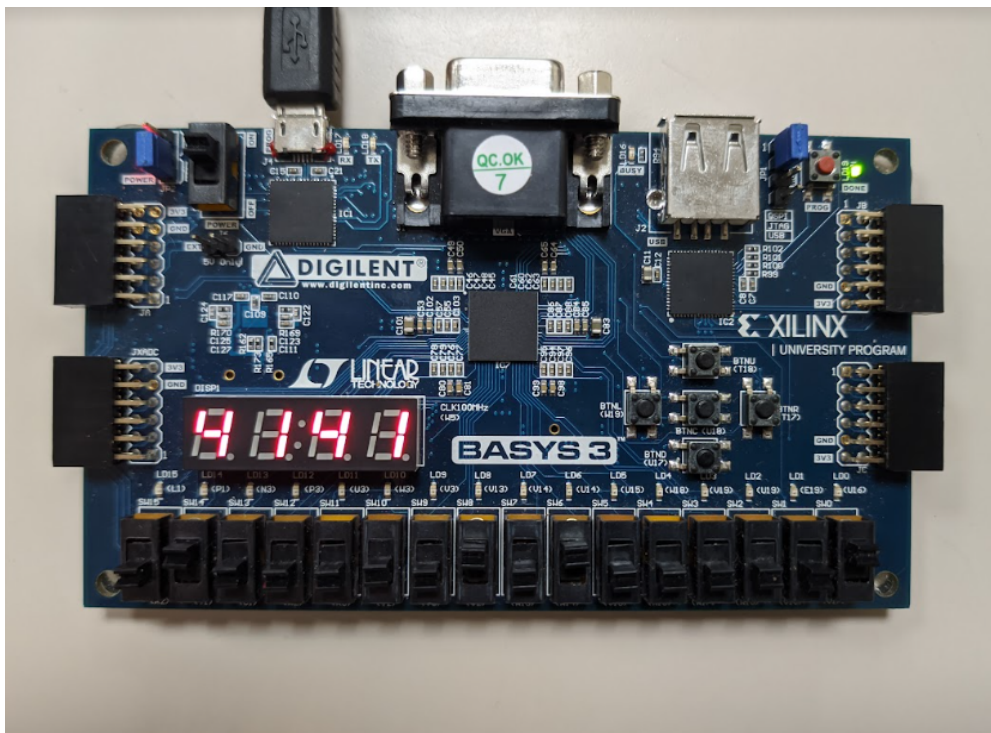
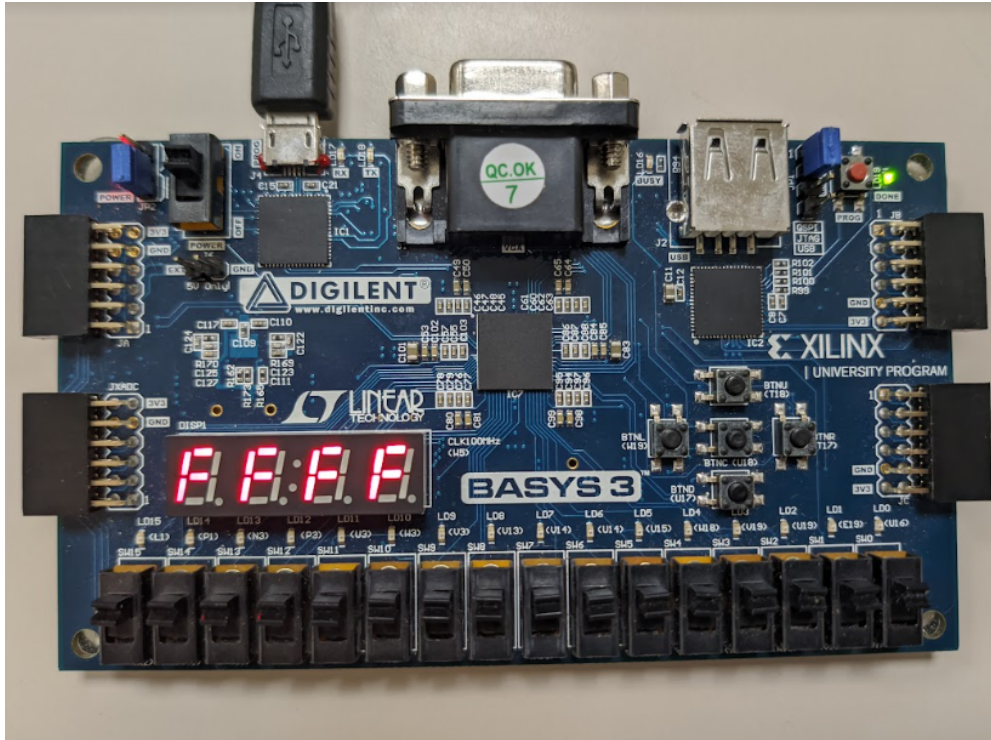
4. SÍNTESI

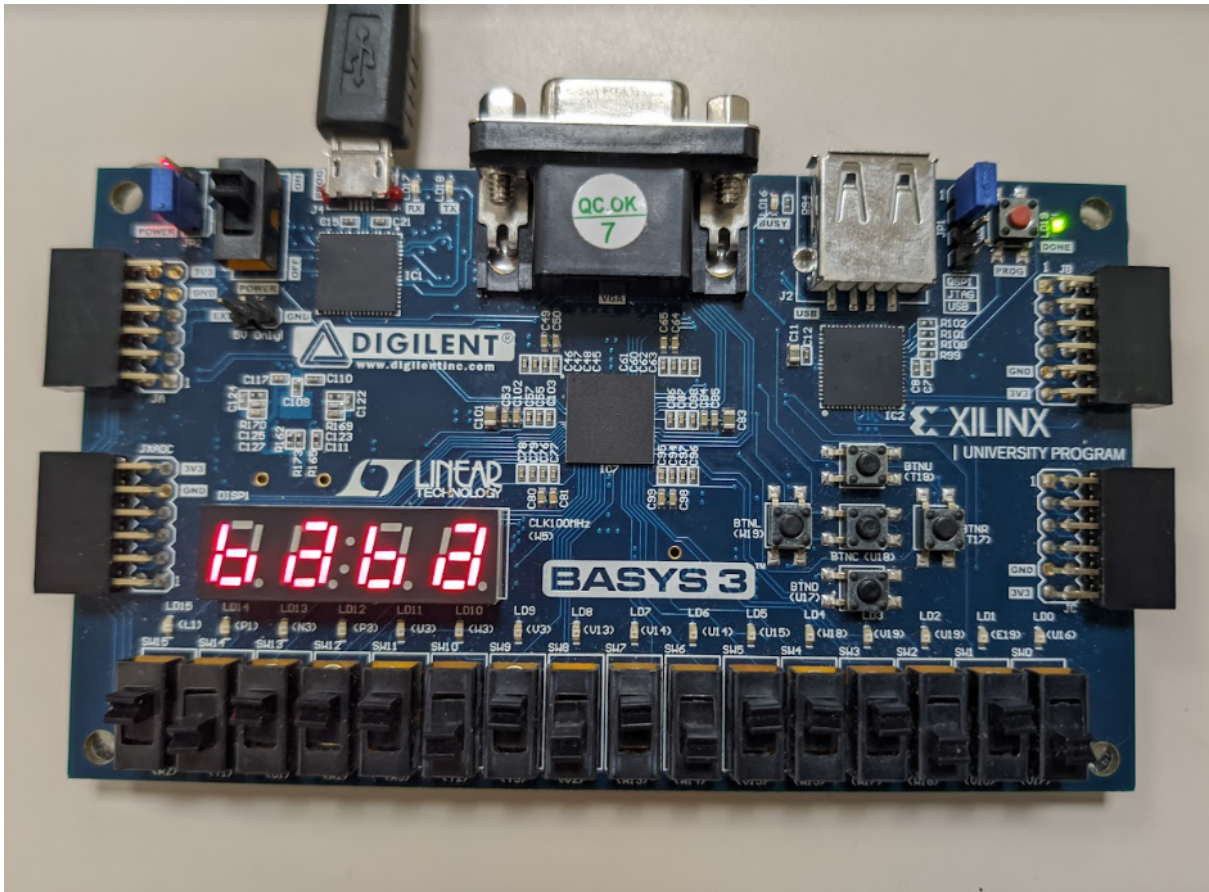
En aquest apartat es mostra el funcionament del nostre programa a nivell de portes lògiques.



5. RESULTATS FINALS

A continuació mostrem les fotos del programa carregades a la placa per validar el seu funcionament.





6. SEGONA PART

Tot el codi roman igual que a la part 1 excepte els fragments presentats a continuació.

Les entrades, sortides i senyals utilitzats han estat:

```
entity p2_sincron2 is
    Port ( sw : in STD_LOGIC_VECTOR (12 downto 0);
          clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          cat : out STD_LOGIC_VECTOR (7 downto 0);
          an : out STD_LOGIC_VECTOR (3 downto 0));
end p2_sincron2;

architecture Behavioral of p2_sincron2 is
    signal div_clk: std_logic;
    signal div_cnt: integer range 0 to 100000;
    signal digit_act: integer range 0 to 3;
    signal sg: std_logic_vector (3 downto 0);
    signal bcdo: std_logic_vector (15 downto 0) := "0000000000000000";
```

No hem aconseguit implementar el convertidor de binary a BCD, no obstant presentem el codi que hem generat per a aquesta fi. Hem intentat implementar l'algoritme "Double Dabble".

```
-- Conversió binari a BCD
double_dabble: process(sw, bcdo)
    variable bcd: std_logic_vector (15 downto 0) := "0000000000000000";
    variable shift: std_logic := '1';
    variable i: integer range 0 to 12 := 12;
    variable j: integer range 0 to 15 := 15;
begin
    while i>0 loop
        if shift = '1' then
            while j>1 loop
                bcd(j) := bcd(j-1);
                bcd(0) := sw(i);
                j := j - 1;
            end loop;
        end if;
        if bcd(3 downto 0) > "0100" then
            bcd := bcd + "11";
            shift := '0';
        elsif bcd(7 downto 4) > "0100" then
            bcd := bcd + "110000";
            shift := '0';
        elsif bcd(11 downto 8) > "0100" then
            bcd := bcd + "1100000000";
            shift := '0';
        elsif bcd(15 downto 12) > "0100" then
            bcd := bcd + "11000000000000";
            shift := '0';
        else
            shift := '1';
        end if;
        i := i - 1;
    end loop;
    bcdo <= bcd;
end process;
```

Modifiquem l'assignació del càtode per tal que es correspongui amb el nombre BCD a representar.

```
-- 3 Valors dels catodes
selector: process(digit_act, sg, bcdo) begin
    -- Assignar conjunt de switches a sg
    case digit_act is
        when 0 =>
            sg <= bcdo(3 downto 0);
        when 1 =>
            sg <= bcdo(7 downto 4);
        when 2 =>
            sg <= bcdo(11 downto 8);
        when 3 =>
            sg <= bcdo(15 downto 12);
    end case;
end process;
```

7. CONCLUSIONS

En aquesta pràctica hem introduït factor temps en el nostre programa a través de l'ús de processos sincrons i el divisor de rellotge que hem emprat per representar els dígit a cada un dels *displays* de 7 segments. També hem utilitzat comandes de VHDL com *case* o *with-select* entre d'altres.